

Feasibility Study

Clario

A Multi-Agent Customer Support Triage and Response System

Date: 25/07/2026

Authors: WEERASEKARA R.V. (230694J), WETTASINGHE V.N. (230701G), WICKRAMARATNA S.I.P. (230703N)

Group ID: Group 23 (Project ID: P04)

Mentor: Dr. Aloka Fernando

Teaching Assistant: Dilanka

Table of Contents

1. Introduction.....	2
1.1 Overview of the Project.....	2
1.2 Objectives of the Project.....	4
1.3 The Need for the Project.....	5
1.4 Overview of Existing Systems and Technologies.....	5
1.5 Scope of the Project.....	7
1.6 Deliverables.....	7
2. Feasibility Study.....	8
2.1 Financial Feasibility.....	8
2.2 Technical Feasibility.....	9
2.3 Resource and Time Feasibility.....	9
2.4 Risk Feasibility.....	10
2.5 Social and Legal Feasibility.....	11
3. Considerations.....	11
4. References.....	12

1. Introduction

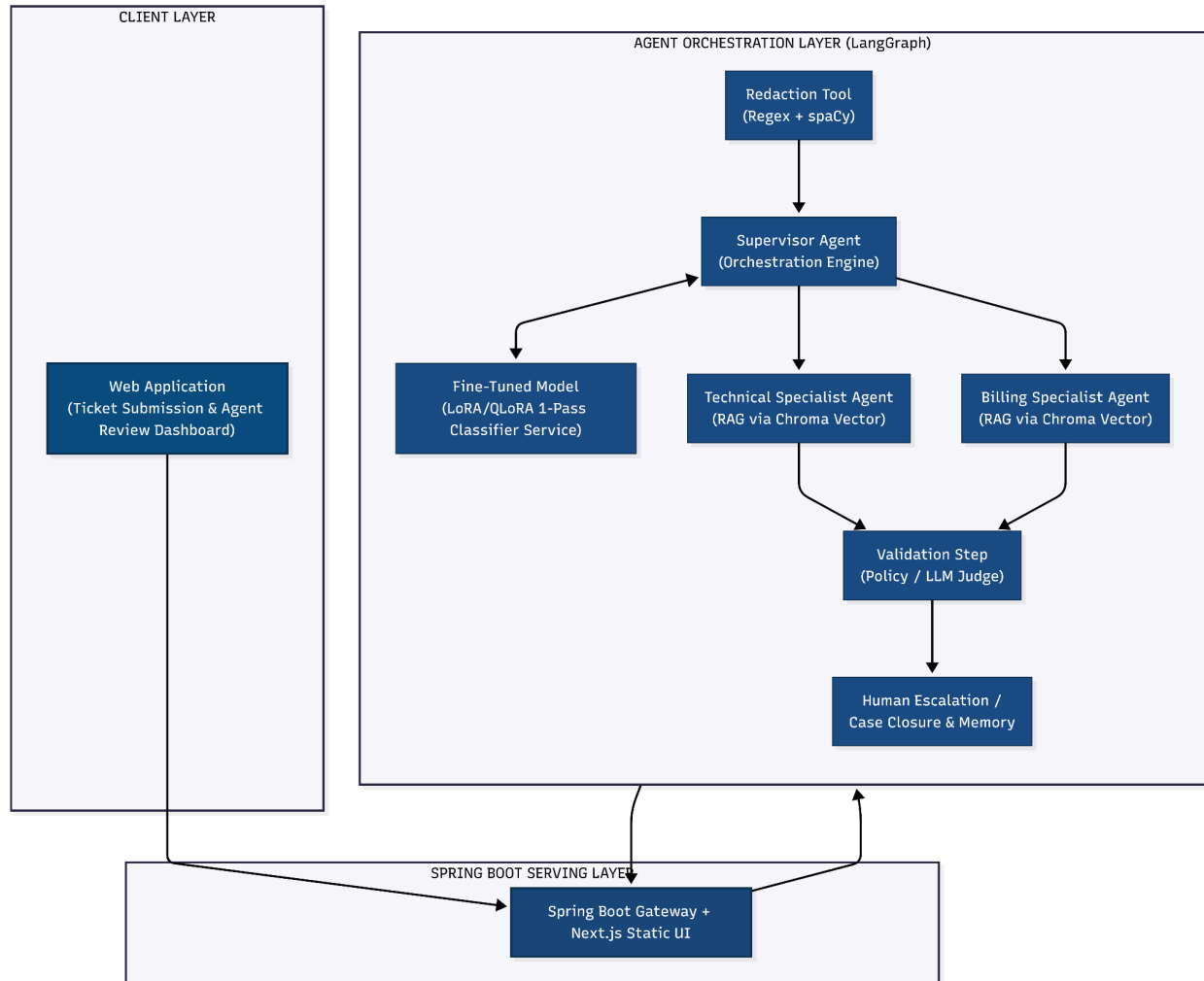
1.1 Overview of the Project

Clario is an automated multi-agent customer support triage and response generation system built on an autonomous, decentralized agent architecture rather than a monolithic classifier pipeline. By moving away from single-model pipelines, Clario enables specialized AI agents to collaborate dynamically under the control of a stateful graph orchestrator.

When an incoming support ticket enters the system, a **Supervisor Agent** inspects the request, extracts intent, language, priority, and sentiment, and decomposes the request into distinct sub-tasks. These sub-tasks are dynamically routed to specialist agents: a **Technical Specialist Agent** and a **Billing Specialist Agent**. Each specialist agent leverages Retrieval-Augmented Generation (RAG) over domain-specific knowledge bases to draft accurate, grounded responses.

To optimize throughput and reduce computational overhead, the Supervisor Agent uses a lightweight, open-weight Large Language Model (LLM) fine-tuned via Low-Rank Adaptation (LoRA/QLoRA). This custom model acts as a single-pass tool that predicts ticket category, priority, and sentiment simultaneously without requiring expensive multi-turn reasoning calls for basic classification.

Data privacy is strictly enforced through a deterministic pre-processing **Redaction Tool** that masks Personally Identifiable Information (PII) before external model requests occur. Drafted responses undergo an automated **Validation Step** before being displayed on a support agent review dashboard. High-complexity, urgent, or sensitive tickets are handed off to human agents alongside complete agent reasoning summaries.



1.2 Objectives of the Project

The key objectives of the project are:

- **Design and implement an autonomous multi-agent orchestration framework** capable of decomposing, routing, and synthesizing responses for customer support tickets.
- **Develop a fast, single-pass classification tool** by fine-tuning an 8B-parameter open-weight LLM using Knowledge Distillation and Parameter-Efficient Fine-Tuning (LoRA/QLoRA).
- **Ensure strict data privacy and regulatory compliance** through deterministic PII redaction (regex + spaCy NER) executed prior to any external model calls.
- **Eliminate model hallucinations and enforce policy grounding** by equipping specialist agents with Retrieval-Augmented Generation (RAG) over curated knowledge bases.
- **Construct an efficient human-in-the-loop handoff mechanism** that supplies human

support representatives with clear contextual summaries and reasoning logs for complex cases.

- **Provide comprehensive empirical evaluation** measuring multi-task classification accuracy, routing accuracy, escalation precision/recall, latency, and per-ticket costs compared to single-LLM baselines.

1.3 The Need for the Project

Customer service operations across enterprise and digital platforms experience significant operational friction due to manual triage bottlenecks. Manually reading every ticket, assessing urgency, evaluating customer sentiment, and drafting replies requires substantial time and labor, while leading to inconsistent resolution quality.

Monolithic models and legacy rule-based keyword routers struggle when encountering multi-topic tickets. For instance, a customer reporting a billing error caused by a technical software glitch requires coordinated domain expertise across both technical and financial operations. Static rules frequently misroute such queries, increasing resolution times and support costs.

Additionally, exposing customer communications directly to external ungrounded LLMs presents serious data privacy risks and potential hallucinated policy commitments. Clario fulfills the operational need for an open, privacy-preserving, transparent, and modular multi-agent support architecture that automates repetitive customer inquiries while maintaining strict policy adherence and human oversight.

1.4 Overview of Existing Systems and Technologies

Commercial helpdesk platforms such as Zendesk (Intelligent Triage) and Freshdesk (Freddy AI) offer automated ticket routing and AI assistance. However, these proprietary solutions operate as closed black-box software with expensive per-agent subscription fees and limited customizability for platform-specific workflows.

Standard academic and industrial approaches often use monolithic single-prompt LLM pipelines. Studies indicate that monolithic prompts suffer from context degradation, high latency, elevated per-token expenses, and higher hallucination rates when processing multi-intent support requests.

Comparative Framework Matrix:

Technological Layer	Selected Framework / Tool	Evaluated Alternatives	Rationale for Selection
Agent Orchestration	LangGraph	CrewAI, AutoGen	Provides explicit state machine control, cyclic graph support, and inspectable node transitions for supervisor-specialist feedback loops.
Classification Engine	Fine-Tuned Llama-3-8B / Mistral-7B (QLoRA)	Direct GPT-4 / Claude-3.5 API calls	Executes single-pass multi-task classification locally at low latency with zero per-token recurring external API costs.
Vector Retrieval (RAG)	ChromaDB	Pinecone, Milvus	Lightweight, open-source embedded vector database offering rapid local search without cloud hosting licensing fees.
Backend & Serving	FastAPI With Spring Boot & PostgreSQL	Flask, MongoDB	High-performance asynchronous python processing paired with structured relational logging for agent execution state.
Data Redaction	Regex + spaCy NER Tool	LLM-based Redaction Agent	Guarantees deterministic, zero-cost PII masking without introducing probabilistic LLM latency or token costs.

1.5 Scope of the Project

System Roles and User Functionalities:

- **Customer / End-User:** Submits support inquiries via the web frontend, views ticket status updates, and receives validated resolution responses.
- **Support Agent / Human Reviewer:** Uses the agent review dashboard to inspect automated classification metrics, review drafted specialist responses, edit suggestions,

and manage escalated tickets.

- **System Administrator:** Manages knowledge base vector collections, reviews operational benchmarks (automation rate, latency, escalation accuracy), and configures fallback thresholds.

In-Scope Boundaries:

- Multi-agent orchestration and conditional routing graph built with LangGraph.
- Single-pass classification tool trained via QLoRA on a small open-weight LLM.
- Knowledge distillation pipeline utilizing Google Gemini API for generating synthetic training labels.
- Deterministic PII masking engine (Regex + spaCy NER) executing prior to LLM processing.
- Specialist agents (Technical & Billing) integrated with local RAG retrieval (ChromaDB).
- Policy and tone compliance checking via an automated LLM-as-judge validation step.
- Historical case memory store embedding resolved tickets for precedent lookup.
- Hybrid Monolithic Architecture

Out-of-Scope Elements:

- Direct integration with commercial CRM/helpdesk products (e.g., Salesforce, Zendesk).
- Training base foundation LLMs from scratch.
- Voice or phone-based conversational support.
- Fully autonomous communication for high-priority, sensitive, or low-confidence tickets without initial human review oversight.
- Production integration with external LMS software (integration with Rysera Stem LMS is maintained strictly as an optional stretch goal).

1.6 Deliverables

The final project deliverables comprise:

1. **Web Application Interface:** A web-based application offering a customer ticket submission interface and an interactive support agent review dashboard.
2. **Containerized Application Architecture:** Docker containers and configuration files unifying the FastAPI backend, LangGraph orchestrator, fine-tuned model service, and PostgreSQL storage layer.
3. **Machine Learning Artifacts:** Trained LoRA/QLoRA adapter weights, data preprocessing scripts, distillation code, and model serving wrappers.
4. **Evaluation Benchmark Suite:** A comprehensive report documenting classification F1-scores, routing accuracy, escalation precision/recall, system latency, per-ticket costs, and single-LLM ablation comparisons.

2. Feasibility Study

2.1 Financial Feasibility

The project is designed to operate within zero-capital resource constraints by strategically utilizing open-source models, permissive frameworks, and free tier cloud computing compute quotas:

- **Compute Budget:** Model fine-tuning and data distillation tasks are executed using free GPU allocations provided by Kaggle Notebooks and Google Colab (NVIDIA T4 / P100 environments).
- **API Expenditure:** Data distillation and baseline comparative evaluations utilize the free tier quota of Google AI Studio (Gemini API).
- **Licensing Costs:** All primary frameworks (Python, PyTorch, LangGraph, FastAPI, PostgreSQL, Docker) are governed by open-source licenses, requiring zero capital expenditure.
- **Storage Infrastructure:** Relational logs and vector indices execute locally without recurring cloud database fees.

Financial Cost Projections:

Expense Category	Commercial Benchmark Costs	Clario Project Expenses
Software Licensing	\$50 - \$150 per agent / month	\$0.00 (Open-Source Stack)
Compute & Training	Dedicated cloud GPU clusters (\$2.00+/hr)	\$0.00 (Kaggle / Colab Free Tier)
API Token Fees	\$0.01 - \$0.03 per support ticket	\$0.00 (Local Single-Pass Classifier + Free Gemini Tier)
Total Capital Expense	Substantial Recurring OpEx	\$0.00

The financial analysis demonstrates complete economic feasibility, validating that the system can be developed, tested, and demonstrated at zero financial cost while delivering substantial potential cost savings to adopting organizations.

2.2 Technical Feasibility

The technical feasibility of Clario is grounded in proven software architecture and state-of-the-art machine learning practices:

1. **Stateful Graph Orchestration:** LangGraph enables cyclic agent workflows with explicit state management, ensuring predictable conditional branching, human-in-the-loop state pausing, and robust error recovery.
2. **Parameter-Efficient Fine-Tuning (QLoRA):** Fine-tuning an 8B open-weight model using 4-bit NormalFloat (NF4) quantization compresses memory usage while preserving model capacity. This allows fine-tuning on standard GPU hardware without memory overflows.
3. **RAG & Vector Search:** Specialist agents perform dense semantic retrieval using sentence embeddings paired with a persistent ChromaDB vector store, providing exact factual grounding from knowledge base snippets.
4. **Deterministic Redaction:** Regex patterns combined with spaCy Named Entity Recognition (NER) provide deterministic data privacy protection before external language model invocations.

2.3 Resource and Time Feasibility

The development plan spans a structured 10-week timeline divided into two parallel tracks (Track A: Fine-Tuning & Data Engineering; Track B: Agent Orchestration & Systems Engineering) that converge into a shared integration and benchmarking track.

Timeline Phase	Track A: Fine-Tuning & ML Engineering	Track B: Orchestration & Systems Integration
Weeks 1 -2 (Jul 18 -Aug 02)	Dataset ingestion (200K Kaggle), EDA, PII Redaction module, held-out test split setup.	Repo configuration, multi-agent state schema design, initial LangGraph structure.
Weeks 3 -5 (Aug 03 -Aug 23)	Gemini knowledge distillation scripts, QLoRA fine-tuning setup, model training on Colab/Kaggle.	Supervisor node implementation, specialist RAG agent development, baseline pipeline testing.
Weeks 6 -7 (Aug 24 -Sep 06)	Held-out test evaluation, metric calculation, FastAPI inference wrapper setup.	Wiring fine-tuned model service into orchestrator graph, fallback testing, Docker containerization.
Weeks 8 -9 (Sep 07 -Sep 20)	Docker Compose stack integration, end-to-end evaluation (latency, cost, ablation studies), results compilation.	

Timeline Phase	Track A: Fine-Tuning & ML Engineering	Track B: Orchestration & Systems Integration
Week 10 (Sep 21 - Sep 27)	System polishing, rehearsal, and final presentation of the project.	

The equal allocation of human resources across three team members ensures all milestones are technically feasible within the allocated timeframe.

2.4 Risk Feasibility

A structured risk analysis identifies potential technical and operational challenges alongside mitigation protocols:

Risk Event	Severity	Likelihood	Mitigation Strategy
Label Noise / Dataset Imbalance	Medium	High	Perform stratified dataset sampling, Gemini-guided distillation for relabeling, and class-weighted loss functions.
RAG Specialist Hallucinations	High	Medium	Enforce top-K snippet grounding, strict prompt guardrails, and an automated LLM-as-judge validation step.
GPU Free Tier Timeouts	Medium	Medium	Implement checkpoint saving, disk caching, retry loops, and 4-bit model quantization to reduce training duration.
Misclassifying Urgent Tickets	High	Low	Implement deterministic safety rules in the Supervisor Agent graph to force immediate escalation for high-urgency keywords.

2.5 Social and Legal Feasibility

Data Privacy & Regulatory Compliance: Clario adheres to global data protection laws (such as GDPR) by executing deterministic PII masking locally prior to model processing. No unredacted personal identifiers are transmitted to external APIs or recorded in vector indices.

Ethics & Governance: The system maintains rigorous human-in-the-loop oversight for all high priority, urgent, or low-confidence support requests. Routine, policy-compliant queries are safely automated, while high-risk or ambiguous drafts are gated at the Human Agent Review Screen for validation, modification, or rejection prior to customer dispatch.

Intellectual Property: All software dependencies, libraries, models, and tools are open-source and governed by permissive licenses (Apache 2.0, MIT, BSD, Llama 3 Community License).

3. Considerations

Key operational and software engineering considerations addressed by the system architecture include:

- **Latency Optimization:** Replacing multi-turn LLM classification prompts with a single-pass fine-tuned local model reduces classification latency to ~200ms, ensuring real-time operational efficiency.
- **Hybrid Monolithic Architecture:** Packaging the frontend static assets (Next.js) directly inside the enterprise gateway (Spring Boot) creates a highly optimized hybrid monolith. This co-location eliminates network serialization latency and browser CORS issues, while the heavy ML and LangGraph engines are isolated into an adjacent local sidecar process.
- **User Interface & Workflow Integration:** The support agent dashboard presents extracted metadata, masked PII spans, retrieved KB context, and draft replies, reducing human decision overhead.
- **Maintainability & Dynamic Updating:** Decoupling specialist domain knowledge into external vector stores allows system documentation to be updated instantly without requiring model retrain cycles.

4. References

1. E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.
2. T. Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.
3. M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" *arXiv preprint arXiv:2503.13657*, 2025.
4. LangChain, Inc., "LangGraph Documentation," 2026. [Online]. Available: <https://langchain-ai.github.io/langgraph/>. (Accessed on Jul. 20, 2026).
5. Meta AI, "Llama 3 Documentation," 2024. [Online]. Available: <https://developer.meta.com/ai/models/llama-3/>. (Accessed on Jul. 20, 2026).
6. Google DeepMind, "Gemini Models Documentation," Google AI Studio, 2026. [Online]. Available: <https://ai.google.dev/docs>. (Accessed on Jul. 20, 2026).
7. Kaggle, "Customer Support Tickets Dataset (200K Records)," Kaggle Datasets, 2024. [Online]. Available: <https://www.kaggle.com/datasets/mirzayasirabdullah07/customer-support-tickets-dataset-200k-records>. (Accessed on Jul. 20, 2026).
8. Zendesk, "Automatically Classifying Tickets with Intelligent Triage," Zendesk Help Center, 2026. [Online]. Available: <https://support.zendesk.com>. (Accessed on Jul. 20, 2026).